

La Salle Computer Society

Backend Development

Challenge

September 2026 - 41st LSCS - Term 1



The Macky Merch API

Welcome to the **technical assessment** for the **Systems and Infrastructure Committee (Backend Track)**. In LSCS, we value the **"You build it, you test it"** philosophy.

This exam is designed to evaluate your fundamental understanding of backend architecture, your ability to handle data, and your discipline in writing automated tests for your own code.

Objective

The **goal of this take-home exam is to assess your ability** to design, implement, test, and document a RESTful API using Node.js, Express.js, and a database of your choice.

Upload your code to a GitHub Repository (public, or private with invites sent to the provided GitHub usernames), then **submit the repo link via the application form before the provided deadline**.

Task Overview

You are tasked with **building a RESTful API using Node.js and Express.js to manage an inventory system** for "Macky Merch" (the official LSCS merchandise).

Your **API must allow a client application to perform standard CRUD** (Create, Read, Update, Delete) operations on a database of merchandise items.

Core Requirements:

1. Technology Stack

- **Backend:** Node.js with Express.js
- **Language:** JavaScript or TypeScript (highly encouraged and will earn bonus points)
- **Database:** Any (e.g., MySQL, PostgreSQL, MongoDB, SQLite, Firebase). A real database is preferred over a local JSON file or in-memory array, but beginners may use either.
- **Testing:** Vitest or Jest
- **Dependencies:** You may use any standard libraries (e.g., mysql2, mongoose, prisma, zod). Please list them in your package.json.

2. Product Data Model & Database Schema

Each product should at least have the following attributes:

- **id** - Unique identifier (auto-increment or UUID, primary key).
- **name** - The name of the product (string, not null).
- **price** - The price of the product (number, not null, positive value).
- **stock** - The quantity available in inventory (number, not null).
- **category** - The product category, e.g., "Clothing", "Accessories" (string, not null).

NOTE: YOU ARE REQUIRED TO ADD MORE ATTRIBUTES. Add at least two (2) custom attributes of your own choosing that you believe a real merchandise system would need (e.g., size, isAvailable, imageUrl, description, createdAt). This tests your ability to think beyond basic instructions.

If using a SQL database (MySQL, PostgreSQL, SQLite), include your CREATE TABLE statement in a file named schema.sql.

3. The API Endpoints

Your API **must implement the following RESTful endpoints**. All requests and responses should be in JSON format (application/json):

Method	Endpoint	Description	Success Response	Error Response
POST	/api/products	Creates a new product. Request body should contain the product's attributes.	201 Created with the newly created product object.	400 Bad Request if validation fails. 500 Internal Server Error for other errors.
GET	/api/products	Retrieves a list of all products.	200 OK with an array of product objects.	500 Internal Server Error.
GET	/api/products/:id	Retrieves a single product by its ID.	200 OK with the product object.	404 Not Found if the product does not exist. 500 Internal Server Error.
PUT	/api/products/:id	Updates an existing product's details by its ID.	200 OK with the updated product object.	400 Bad Request for invalid data. 404 Not Found if

Method	Endpoint	Description	Success Response	Error Response
		Request body can contain any subset of the product's attributes.		the product does not exist. 500 Internal Server Error.
DELETE	/api/products/:id	Deletes a product by its ID.	200 OK with a success message (e.g., { "message": "Product deleted successfully." }).	404 Not Found if the product does not exist. 500 Internal Server Error.

4. Input Validation & Error Handling

- **Implement server-side validation** for incoming data on POST and PUT requests (e.g., name should be a non-empty string, price should be a positive number, etc.).
- Your API **should handle errors gracefully and return appropriate HTTP status codes** with informative JSON error messages.

5. Automated Testing (TDD)

In the Systems and Infrastructure committee, Backend Engineers act as their own QA ("**You build it, you test it**").

- You must write at least two (2) automated tests using Vitest or Jest.
- Example: One test verifying that GET /api/products returns a 200 status, and one test verifying that a POST request with missing fields returns a 400 error.
- Ensure your package.json includes a working test script.

6. Documentation

You must include a README.md file in your repository that contains:

1. **Setup Instructions:** Exact commands to install dependencies, set up the database, and run your server and tests locally.
2. **Architectural Explanation:** Briefly explain why you chose your folder structure and why you chose your specific database.
3. **Challenges Faced:** One bug or hurdle you encountered while building this and how you solved it.

7. Bonus Features (Optional)

Implementing any of the following will be highly regarded:

- **Pagination:** Support GET /api/products?page=1&limit=5 to paginate results.
- **Containerization:** Include a working Dockerfile so your API can be spun up instantly.
- **Git Workflow:** Use Feature Branches and merge via Pull Requests instead of pushing

directly to main.

Submission Checklist

Before submitting, verify the following:

- ☐ All 5 CRUD endpoints are functional.
- ☐ At least 2 custom product attributes are added.
- ☐ At least 2 automated tests pass successfully.
- ☐ README.md is included with setup instructions, architectural explanation, and challenges.
- ☐ package.json has working start (or dev) and test scripts.
- ☐ node_modules/ and .env files are NOT uploaded (use .gitignore).
- ☐ If using SQL: schema.sql file is included.

Evaluation Criteria

- **Correctness & Functionality (30%)** - Does the API work as specified? Do all CRUD endpoints function correctly? Are custom attributes implemented?
- **Code Quality & Structure (20%)** - Is the code clean, well-structured, readable, and maintainable? (e.g., routes separated from controllers/models)
- **API Design & Error Handling (15%)** - is the API RESTful and intuitive? Does it return proper HTTP status codes and informative error messages?
- **Database Schema (10%)** - Is the schema well-designed with appropriate data types, constraints, and custom attributes?
- **Testing (15%)** - do the automated tests run successfully? Do they test meaningful logic beyond trivial assertions?
- **Documentation & Comprehension (10%)** - Are the setup instructions comprehensive, and does the applicant demonstrate full ownership and understanding of the architecture they built?